Folios — Upgrade Log

FOLIOS · FOLIOSHQ.COM

LAST UPDATED: 2026-06-21 (WIN LOG + THE FRIDAY PIP WRAP — YOUR TRACK RECORD, KEPT)

2026-06-21 — Your track record: the Win log + the Friday Pip Wrap

What I built: Two connected things that finally make Folios remember the good — not just the to-dos. A **Win log** (your brag file): the things that went right get saved so they're still there at review time, instead of evaporating the moment you move on. And a **Friday Pip Wrap** on Home: a tidy week-in-review of what you actually got done.

Problem it solves: You're measured on keeping suppliers happy and on not dropping things — but the *evidence* of a strong week (promises kept, projects moved, fires put out) was nowhere. When review season comes, you're reconstructing it from memory. Now it accumulates on its own.

What changed:

- **Promises kept** — Folios now tracks your commitments kept-on-time vs. slipped, and shows the rate in Settings → Pip (next to "is Folios earning its keep?"). It's the honest scorecard on whether you keep your word.
- **Win log** — confirmed wins persist in Settings → Pip. Log anything by hand, and the Friday Wrap offers **one-tap wins** for things it spots automatically (a project you completed, a promise you kept on time).
- **Friday Pip Wrap** — every Friday a card appears on Home with the week: promises kept/slipped, accounts you met vs. ones that went quiet, projects that moved, wins logged. It's built **for free** (no AI) — and if you want Pip's read on *how* the week went, one tap adds a short reflective paragraph.

What you see today: A "Promises kept" stat and a Win log in Settings → Pip, and — on Fridays — a Pip Wrap card on Home.

Why it matters: This is the half of the picture Folios was missing. It already nags you about what's overdue; now it also banks what went right. That compounds into a real promotion case — and into Pip understanding your week, not just your backlog.

(EVERYTHING HERE IS ABOUT YOUR OWN WORK — PROMISES, PROJECTS, WINS. NO BUSINESS NUMBERS ARE INVOLVED, BY DESIGN.)

2026-06-21 — Folios owns the team request tracker (the Team Sheet)

What I built: Your team's Tuesday meeting runs off a shared Excel request tracker — and you've been keeping it updated by hand while also tracking the same projects in Folios. That double-entry was dying: lots of projects in Folios never made it onto the sheet. Now Folios **generates the sheet for you.** Flip "Track on team sheet" on any Gauge project and it shows up in a new **Gauge → Team Sheet** view, laid out in your spreadsheet's exact column order. One tap copies the rows that changed since you last exported, ready to paste straight into Excel.

Problem it solves: You can't maintain two systems, so the sheet drifted. Make Folios the master and the sheet an output, and the drift problem disappears — the spreadsheet is always a copy of what's already in Folios.

What changed:

- Any project can be flagged for the team sheet, with a few tracker-specific fields (email thread link, connection/integration macro dates). Everything else — priority, request date, owner, supplier, initiative, required completion date, latest comment — comes from the project automatically.
- The **Team Sheet view** is a clean, screen-shareable grid for the Tuesday meeting, with unsynced rows marked.
- **"Copy unsynced rows"** puts exactly the changed rows on your clipboard as tab-separated lines that paste into the spreadsheet's cells.
- A gentle **nudge on Home** (Monday afternoon / Tuesday morning) reminds you when tracked projects have changes that aren't on the sheet yet — one tap to

copy them.

What you see today: A "Team Sheet" tab in Gauge. Turn it on for the projects your team tracks, and the weekly sheet update becomes copy-and-paste instead of hand-maintenance.

Why it matters: It closes the last "I maintain the same thing in two places" gap — one of the things you said you were worst at. The sheet stays current without you babysitting it, and nothing in Folios contradicts what the team sees on Tuesday.

(ONE COLUMN STAYS MANUAL ON PURPOSE: "# OF SHOPS" IS A CUSTOMER-COUNT FIGURE, SO IT NEVER LIVES IN FOLIOS — THE EXPORT LEAVES THAT CELL BLANK FOR YOU TO FILL IN EXCEL. THAT'S THE DATA LINE DOING ITS JOB.)

2026-06-19 — Pip remembers the right thing, not just the recent thing

What I built: Until now, when Pip pulled up context on an account, it only ever saw the *latest* handful of meetings. Anything older — a decision you made six months ago, a constraint someone mentioned once and never again — was effectively invisible unless it happened to be recent. I gave Pip **semantic recall**: it can now find the most *relevant* past notes by meaning, not by date. Ask "what did we decide about the invoice-feed integration?" and Pip surfaces that conversation even if it was half a year and forty meetings ago.

Problem it solves: Your real institutional memory isn't the last three meetings — it's spread across everything you've ever written down. "Recency" is a bad proxy for "relevance." This is the start of Folios being the portable brain it's meant to be: the longer you use it, the more it remembers, and the more it can connect today's question to something you said long ago.

What changed:

- Folios quietly keeps a meaning-based index of **your own notes** — meeting notes, Pip's meeting summaries, the per-project notes you type in meetings, and account updates. A once-a-day background pass adds anything new; it never re-indexes something that hasn't changed, so it costs effectively nothing to keep current.

- When you ask Pip a question, Folios matches it against that index and slips the most relevant past notes into Pip's context — scoped to the account you're asking about, or across your whole book for "did we ever decide…" questions. It flows through the one shared context builder every Pip surface uses, so it just works in chat.
- **Your data stays yours.** Only your own notes are indexed, locked to your account with a database-level owner check, and the data line holds — no revenue, volumes, or shop lists are ever embedded (your own words go in verbatim; Pip's summaries were already scrubbed of numbers when written).

What you see today: Pip gets noticeably better at "remember when…" questions — it can reach back past the recent meetings to the thing that actually matters. Everyday briefs and summaries look the same; the difference shows up when the useful context is old. *(Recall switches on once an embeddings key is configured; until then Pip simply falls back to recent-meetings context as before.)*

Why it matters: This is the memory that compounds. Every note you take makes the next "what did we decide about X" answer better — and it travels with you. It's the foundation for deeper portable-brain features (cross-account pattern spotting, "since you were here" deltas) down the line.

2026-06-19 — Pip can look things up mid-answer (the chat agent loop)

What I built: Until now, when you asked Pip something in chat, it answered in one shot from whatever was already loaded in front of it — and if it wanted to *do* something (log a meeting, set a follow-up), it handed that action to the app to run. It couldn't pause mid-thought to go *find* more. I gave Pip a real **agent loop**: it can now reach for a few read-only lookups while it's thinking — pull up one account in depth, scan the whole book for what's overdue or stalled or waiting on someone, or search your meeting notes for a topic — get the answer back, and keep reasoning before it replies. So "find the stalled project and draft the chase note for it" happens in a single turn instead of you having to do the finding yourself.

Problem it solves: Pip only knew what the screen happened to send it. Ask about an account it didn't have loaded, or "what's slipping across everything," and it had to ask you to narrow down. Now it can go look — within your own data — and answer.

What changed:

- Three new **read-only** lookups Pip can call mid-answer: deep-read one account, find open/overdue/stalled/waiting work across all your accounts, and search your own notes and summaries. They only ever *read*, only your own data, and write nothing.
- Pip's existing actions (logging a meeting, creating an item, setting health, etc.) are **unchanged** — they still come to you as a card to approve, exactly as before. The loop never commits anything on its own.
- Tight limits so it can't run away: at most a few lookups per question, full cost still shown on the spend tile, and if Pip doesn't need to look anything up, the answer is a single step just like before.

What you see today: Chat feels the same for simple questions, and noticeably smarter for "go find X" ones — Pip can pull the detail itself instead of asking you to. Anything that changes your data still asks first.

Why it matters: It's the step from "Pip answers from what's in front of it" to "Pip can go look, then answer" — the foundation for Pip doing multi-step work for you in one ask, while keeping every actual change behind your approval.

2026-06-19 — The Monday 1:1 pack (never walk in flat-footed)

What I built: A prep sheet for your Monday 1:1 that Folios builds for you, so nothing your boss raises is a surprise. It shows up on the Home screen Monday morning (with a Sunday-evening heads-up) and opens into the full pack inside that 1:1's hub. It reads top to bottom: a one-line read from Pip, then **your word** (the commitments you made — each marked Kept, Slipped, or Open, with the account), then **the boss's open asks, already answered** (Pip pulls what the boss asked about from your last 1:1's notes and attaches where each one stands now), then **what moved this week, by account**, then **who has the ball** — what you owe versus what's owed to you.

Problem it solves: Your biggest fear is being flat-footed — forgetting something you said you'd do, or getting asked "where are we on X" and not having the answer ready. Until now that prep was in your head (or scattered across accounts) every Monday morning. The pack assembles it for you from what's already in Folios.

What changed: Almost the entire pack is built from data you already capture — commitments, meetings, Gauge updates, waiting-ons — so it's free and always current. The only part that needs Pip is the short read and pulling the boss's asks out of your last 1:1's notes, and that's **one model call a week**: the result is saved and only rebuilt when something in the week actually changes, so a quiet week costs nothing. There's **no new step for you** — Pip reads the notes you already type; it doesn't ask you to tag anything. And it stays on the data line: asks and statuses are directional ("waiting on legal," "moving"), never numbers, and your raw notes are read, never edited.

What you see today: On Monday, a "Monday 1:1 Pack" card leads your Home screen with the read and a few count chips (slipped / open / kept / boss asks); tap it to open the full pack in the 1:1. Any day, the full pack lives at the top of that 1:1's hub.

Why it matters: This is the first of the "shine" features — it's not plumbing, it's the thing that makes you look on top of your book in front of your boss every single week, with zero extra effort.

2026-06-19 — Pip recomputes on real events, not on a clock

What I built: Pip keeps a short "state" read for every account (where things stand, momentum, risks) that the app uses without paying for a fresh AI call each time you open a screen. That state used to be refreshed on **timers** — a background pass every few hours over your most-active accounts, plus another sweep every time you opened Pip chat — whether or not anything had actually changed. Most of those refreshes re-described an account that hadn't moved since the morning: money spent to learn nothing. I rewired it so a refresh fires **only when an account genuinely changed** — a meeting logged or summarized, a task added, closed, or edited — and never on a clock.

Problem it solves: It was a quiet, steady drain on the AI budget (Chris flagged "I fly through tokens now"), and it had a blind spot: the old gate only noticed when a *meeting* was logged, so closing a task or editing one never freshened Pip's read. The new design fixes both — it spends only on real change, and it now catches task changes the old one missed.

What changed (two gates, belt-and-suspenders):

- **A cheap front gate (in the app):** when your account, meeting, and task data loads or updates live, the app figures out which accounts have a signal newer than the last time Pip looked at them, and only those get queued. Untouched accounts are never sent. The old "every few hours" and "every time you open chat" timers are gone.
- **A precise back gate (on the server):** for each account that *is* sent, the server computes a small fingerprint of the account's real content and **skips the AI call entirely if the fingerprint is identical** to last time — so even a false alarm from the front gate costs nothing. The fingerprint is built only from stable facts (dates, counts, ids), never from "3 days ago"-style text, so it doesn't drift on its own as the clock ticks. An automated test locks that property in place.
- **The structured context Pip builds for each account is now saved** alongside its state — a durable record of "what Pip knew," and the groundwork for smarter recall later.
- The manual **"resync Pip memory"** button still forces a fresh read on demand, ignoring both gates.

What you see today: Nothing looks different — Pip's reads are the same quality, and actually a little fresher (closing a task now updates his read, which it didn't before). The difference is on the bill: the per-account refresh line should drop by roughly **70–90%**, with no loss of freshness.

Why it matters: It's the difference between an assistant that re-reads the whole book every few hours out of habit and one that looks again only when something actually happened. Same intelligence, a fraction of the cost — and the saved structured context is the foundation for Pip's next memory upgrades.

2026-06-18 — One home for tasks: folio_tasks is now canonical

What I built: Folios had two places that stored "tasks." Loose action items lived in one table (`folio_tasks`); the steps and to-dos *inside* a Gauge project lived in a different place — a list embedded on the project itself (`gauge_projects.stages`). Two stores meant the kanban board and the flat task queue could quietly disagree, and an edit made in one view might not show in

another. I merged them: **every task — loose or inside a project — now lives in `folio_tasks`, the single source of truth.** The old embedded list is kept untouched as a frozen backup, but nothing reads or writes it anymore.

Problem it solves: The split was the root of a whole class of "my edit didn't stick" / "this view shows different tasks than that view" confusion, and it forced every new Pip feature to be wired into two places. One store kills that drift permanently.

What changed:

- **The data:** all 174 project tasks/steps were copied out of the embedded lists into `folio_tasks`, with their completion, assignees, due dates, external-contact flags, blocked reasons, and ordering preserved exactly. Verified row-for-row: per-project task counts and completed-counts match the backup with zero discrepancies. `folio_tasks` gained the columns it needed to hold everything the old format did (external-contact fields, blocked reason, sub-steps, ordering).
- **The reading:** every screen that shows project work — the Gauge board, the account Projects tab, Home's "in flight" cards, the leader rollup, the calendar, and everything Pip reads for briefs and summaries — now reads tasks from `folio_tasks` (projects get their tasks attached automatically when loaded).
- **The writing:** every place you edit project work — the project checklist editor, the kanban board, the meeting-hub task list, the project builder, "escalate an item to a project," and Pip's summarize-and-file flow — now writes to `folio_tasks`.
- The old embedded `stages` list is frozen as a read-only backup (not deleted), so nothing is lost and the change is reversible.

What you see today: Nothing looks different — that's the point. Your projects, tasks, checkmarks, assignees, and progress bars are all exactly where they were. Under the hood they now come from one place, so the board, the queue, the account view, and Pip can never disagree about what's on a project again.

Why it matters: A single task store is the foundation for everything task-related getting more reliable. "Who's holding the ball," commitment tracking, the team request queue, and Pip's read of project work all get simpler and more trustworthy because there's exactly one answer to "what tasks does this project have."

Plain-English log of major upgrades shipped to Folios. Date, time, and a short explanation written in terms anyone can read — not technical release notes.

For the technical changelog with full release detail, see changelog.md. For day-to-day commit history, see `git log` on the production branch.

What gets logged here: major feature shipments, schema migrations, architectural changes, anything that meaningfully changes what Folios *does* or *is*. Not bug fixes, styling tweaks, or doc-only updates — those live in git history.

2026-06-10 — Pip's new 3D form: "Bold Hex"

What I built: Pip got a real body. His mascot form — the two glowing orb circles — is replaced (at medium and large sizes) by a 3D hex-lattice figure: two spheres covered in hexagonal tiles, suspended inside a slowly turning, organically twisted hexagonal ring. The whole figure breathes as one thing on a 2.4-second cycle.

Problem it solves: The original Pip orb was two CSS circles with a glow — simple, recognizable, but not distinctive. The "Bold Hex" design gives Pip a proper visual identity that's alive and unmistakable. More practically: the design was being spec'd and tweaked across sessions, and there was no guarantee a future change wouldn't silently drift the parameters. Now the design is locked in code: one automated test compares a numeric hash of every computed coordinate against a frozen expected value. If any geometry changes — ring radius, breath rate, tilt angle, anything — the CI build fails loudly and names the parameter that changed. You'll never get an accidental redesign.

What changed:

- `src/lib/pip3dGeometry.js` — pure math module. All 20+ locked parameters live here in a frozen constant. Every coordinate for every animation frame is computed here, deterministically, from that constant.
- `src/components/PipOrb3D.jsx` — SVG renderer. Takes what the geometry module computes and writes it to the DOM per-frame using a single shared animation loop. No React re-renders per frame; all instances share one `requestAnimationFrame`.

- `src/components/PipMark.jsx` — the canonical Pip component. `lg`, `xl`, `xxl` sizes now render the 3D scene; `xs`, `sm`, `md` stay as the classic two-circle form (hexes mush at small sizes — intentional).
- `index.html` — new `--accent-hi` CSS token (the brightest hex face color) added to all four palette blocks: work dark, work light, life dark, life light. The 3D scene reads only CSS vars so it re-skins automatically when the accent changes.
- `src/lib/pip3dGeometry.test.js` — the drift lock. Asserts every locked parameter by name, checks the spec is immutable, and hashes the full per-frame output at two time values against expected constants.

What you see today: Open Home and the large Pip orb at the center is the 3D hex-ring figure, turning and breathing. Any other `lg` / `xl` / `xxl` Pip instance (error screens, the cadence "Pip ready" indicator, the loading screen) shows the same figure. Small Pip dots — nav indicators, inline "ask Pip" buttons, meeting icons — are unchanged.

Why it matters: Pip now looks like Pip. The visual identity is locked by an automated test so it's maintenance-free — future developers (or Patch) can't accidentally drift it. The whole scene is drawn with CSS color tokens so it re-themes for free: switch to Life mode and Pip's ring and spheres go from teal to dusty orange without touching the renderer.

2026-06-10 — Pip gets the interview: he finally knows the job

What I built: The deep interview about how the job actually works — what OEC suppliers pay for, how success is measured ("are my suppliers happy"), the audit workflow, the team (boss, analyst, admin), the weekly rhythm, the known failure modes, and which accounts are NOT the user's relationships — is now distilled into a durable "operating context" that rides ahead of Pip's profile in **every** surface: chat, summaries, briefs, the nightly operator run, and question generation.

Why it matters: Pip's biggest weakness was scope — "he doesn't have enough info to truly answer." Now every Pip call starts from the same ground truth the user carries in

his head, including the hard data-line rule and the instruction to stop nudging relationship upkeep on MSO accounts he doesn't own.

2026-06-10 — Pip's questions grow up: guess-first, one-tap, with receipts

What I built: Three upgrades to how Pip learns. **Terminology questions now lead with Pip's guess** — instead of "what is Fuse5?" he says "'Fuse5' — my read: a parts system they run. Am I right?" with a one-tap **"✓ Right — lock it in"** button; only corrections need typing. Questions are **ranked by how much confusion a term actually causes** (the ones appearing most get asked first). **Every answer now shows a receipt** — "Locked in ✦ Pip reads it that way everywhere now" — and the Catch Up session shows a live "5 left · 3 answered" counter so teaching Pip feels like progress, not a void. Also: **one internal cadence can now span multiple departments** (pick extra departments when setting it up; the meeting roster shows everyone), and the **notes editor** finally auto-capitalizes sentence starts and indents sub-bullets visibly deeper.

The data line, now enforced inside Pip himself: the question generators carry a hard rule — never ask for revenue, volumes, customer counts, rosters, pricing, or contract terms; directional questions only. The assistant is designed not to solicit company data, and that's now true in the prompts, not just the policy.

Also fixed along the way: multi-account *task* cadences had been silently broken (the app wrote a database column that never existed) — repairing the schema for multi-department meetings fixed those too.

Why it matters: You said you genuinely answer Pip's questions but never knew what they did. Now the common case is one tap, the order reflects what's actually confusing him, and every answer visibly lands.

2026-06-10 — The two-brain bridge: your work day finally reaches Folios

What I built: A digest handoff between your work Claude (which sees your email and Teams) and Folios (which can't, by design). Two matched halves: a **prompt you add**

to your work-Claude routine that makes it output a sanitized "Folios digest" — commitments you made, things you're waiting on, threads gone quiet, notable exchanges — and a **paste box in Folios** (Home → Quick capture → "Paste work digest") that files every line onto the right account in one tap: commitments become tracked commitments, waiting-ons get a holder and a clock, touchpoints land in account history.

Problem it solves: Most of the job happens in email and Teams and never reached Folios — so Pip's picture was thin, accounts looked colder than they were, and promises made in writing lived only in memory. This was THE capture gap.

The data line, enforced at the source: the work-Claude prompt's first rule forbids revenue figures, volumes, customer counts, shop lists, pricing, and contract terms — the digest arrives already clean, Folios parses it without any AI call, and you review every row before it's filed. Documented in data-handling and AI-governance docs.

What you see today: Run your morning email report at work, paste the digest block into Folios, tap "File it all ✦" — and your accounts, Home ledger, and Pip all know what your inbox knows. Qualitatively, never numerically.

Why it matters: You were the manual sync layer between two AIs. Now it's one paste — and everything downstream (the check-in, Your word, the operator report, account health) runs on a full picture instead of a third of one.

2026-06-10 — The morning check-in, "Your word," and Pip takes center stage on mobile

What I built: Three connected changes to how the day starts. First, the **morning check-in**: before you read Pip's overnight report, he asks up to three one-tap verification questions — "that audit was due Friday, did it land?" / "still stuck on Danny? He's had it 12 days" / "that Tuesday draft was never summarized — still needed?" Each answer immediately fixes the data underneath (marks the item done, clears the hold, routes you to the draft) with a visible receipt. Second, **"Your word"** — one card near the top of Home with both sides of your ledger: what you owe (commitments due or slipping, with Done/Snooze) and what they owe you (everything blocked on someone, with chase notes). Third, **Pip is now front and center in the**

mobile bottom nav — his living orb sits in the middle slot; Commitments gave up the slot and lives on inside "Your word."

Problem it solves: The report used to declare things at full confidence on stale data (the "All Star Monday" — work done over the weekend, just unmarked, reported as a fire). And the things that define being flat-footed — forgotten promises, silent holds — had no single home. And the assistant the app is named for wasn't reachable from the phone nav at all.

What you see today: Open Folios in the morning → Pip's check-in card (when he has something to verify), then Your word, then today's calls, then the corrected report. Duplicates are gone — drafted follow-ups and the question-of-the-day no longer repeat below the report on operator mornings. On your phone, the middle nav button is Pip himself.

Why it matters: "Flat-footed = I forgot something I said I'd do." The morning sequence is now built around exactly that — verify, keep your word, then read the day. The check-in costs nothing to run (no AI calls — it's computed from your own data) and ten seconds to answer.

2026-06-10 — Waiting-on layer + the account Overview becomes a strategic face

What I built: Every project and task can now record **who's holding the ball** — "waiting on Danny since June 4" — set from the project editor or task panel with one picker. Everything blocked on someone else rolls up to a **"They owe you"** card on Home, oldest holds first, each with a one-tap **chase note** copied to your clipboard ready for email or Teams. And the account Overview tab was rebuilt as the account's strategic face: big meeting CTAs on top (start/log, cadence hub, history), the account's projects in flight with status + ball-holder + latest pulse, your open commitments, and the last conversation — with all the existing depth intact below and in the tabs.

Problem it solves: Stalled projects are what anger top suppliers, and stalls usually mean someone else — a POC gone quiet, the product team, the admin — has the ball. That dimension was never tracked, so chasing depended on memory. Meanwhile the account page buried its most-used actions under everything else.

What you see today: Set "Waiting on" on any project or task → an hourglass chip appears on the Gauge card (red past 10 days), it shows on the account Overview per project, Pip sees it in chat and summarize, and Home tells you who owes you what every morning with the chase note one tap away. Account pages open to action buttons and the strategic picture first. The account tabs also scroll on phones now instead of clipping.

Why it matters: "Who's holding the ball" was the single missing dimension behind the projects-stall problem — now it's first-class, visible everywhere, and chaseable in one tap.

2026-06-10 — Summarize precision: Pip recognizes your world and stops inventing tasks

What I built: Three changes to how Pip turns meeting notes into a plan. First, he now carries a **people directory** of everyone you already know — every contact on every account, your partners' people, your internal teammates — so mentioning a name from another account no longer makes him suggest them as a "new contact." Second, his task extraction was retuned from volume to **precision**: notes are treated as a journal, and only genuine commitments, direct asks, and explicitly marked lines become tasks — "no tasks" is now an acceptable answer. Third, **receipts**: the plan preview shows a small "What Pip used" note naming the stored knowledge he actually applied (a term you taught him, a person he recognized, an update event he connected).

Problem it solves: The two most common summarize complaints — phantom contact suggestions for people already known, and a flood of manufactured tasks from informational notes that had to be deleted (or worse, got accepted wrong when rushed).

What you see today: Summarize a meeting → fewer, more correct task rows; people you already know anywhere in your world are never suggested as new contacts; and a small ✦ note in the preview showing what taught knowledge contributed.

Also in this batch — the recap now streams live. Hitting "End & Summarize" no longer means staring at a spinner for up to a minute: Pip's written recap appears word-by-word within a second or two while he structures the plan behind it, and the plan

modal opens the moment it's ready. And inside the plan modal, new tasks are now grouped under the project they'll land on, so a mis-routed task is visible at a glance instead of discovered later.

Why it matters: Wrong rows accepted in a rush poison everything downstream. Precision at the source is what makes the rest of the system trustworthy — and receipts are the proof that teaching Pip pays.

2026-06-10 — Work / Life mode: Folios becomes your whole-day assistant

What I built: A second mode alongside the work mode — a personal assistant side of the app (dusty-blue palette, dusty-orange Pip) for appointments, important dates with escalating reminder ladders, and a honey-do list. A single toggle switches the whole app between Work and Life.

Problem it solves: The app only knew about your job. Everything outside work — dentist appointments, your anniversary, the fence that needs fixing — lived in your head or scattered across calendar apps. This gives Pip a place to carry the personal side of your life too, eventually unifying into one morning read.

What changed:

- **Mode toggle** in the desktop rail and the mobile header flips between Work (green) and Life (dusty blue + orange Pip). The app recolors instantly — same CSS token system as the light/dark switch.
- **Life Home** has three sections: Upcoming appointments and events, a honey-do list prioritized by how long it's been open and how complex the job is, and the soccer card (moved from work Home).
- **VIP heads-up ladder** — mark an event as important (anniversary, spouse's birthday, Christmas) and Pip escalates the nudge over time: first a soft heads-up three weeks out, then harder reminders at one week, three days, one day, and the day itself. Set it once; Pip carries it forward every year.
- **Work mode is byte-for-byte unchanged.** Life is entirely additive.

What you see today: Tap the toggle → the app goes dusty blue, Pip's orb goes orange, and you see your personal calendar/tasks. Tap back → full work mode, green,

all your accounts.

Why it matters: "Folios is my portable brain" means *your whole brain*, not just the work part. This is the foundation; Phase 2 (planned) turns honey-do into a Pip coaching session — how to do the job, what to buy, step by step.

2026-06-10 — Mobile Home redesigned as a structured hub

What I built: A new mobile Home screen that replaces a single wall-of-text Pip card with a set of compact, scannable section cards.

Problem it solves: On a phone, the previous Home showed the full operator report as one long prose block — you had to scroll through all of it to find what you needed. Accounts, fires, upcoming calls, and wins were buried in the same paragraph.

What changed:

- A compact **Pip glance card** at the top shows the one-line headline, scan-able count chips (fires / watches / wins), and a "Show details" expander for the full read.
- Below it, **Today / This Week / Good News / Pattern** each get their own card with a tinted header strip and stacked rows — each row is one account, tappable.
- An **"On the Calendar" card** shows any meetings scheduled for today.
- The old redundant narrative panels (Burning / Calls / Loose / Ahead) are hidden when the operator report has content, so nothing repeats.

What you see today: Open the app on your phone → a tight stack of clearly labeled cards instead of a prose dump. Today's fires are Today. The week's watches are This Week. No scrolling to find the bit that's relevant to the next fifteen minutes.

Why it matters: The morning brief is only useful if you can actually read it on your phone while you're starting your day. This makes it work on a four-inch screen.

2026-06-10 — Split-screen meeting mode: notes that know which project they belong to

What I built: The full-screen meeting room is now split down the middle. Your projects live on the left — tap one "discussed" and it opens its own note field right on the card — and your general notes live on the right. On the phone, a Notes / Projects toggle gives each side the whole screen, one at a time. Adding a new contact mid-meeting stays one tap away in the People section.

Problem it solves: All meeting notes used to land in one big blob, and Pip had to guess which scribble belonged to which project — so tasks sometimes got routed to the wrong place, or to no place. Now notes typed on a project's card carry certain provenance: Pip knows those words are about *that* project.

What changed:

- New `project_notes` storage on every meeting — one slot per project, kept separate from the general notes.
- Pip's summarize call receives the per-project notes as their own labeled blocks and is instructed to route a project's action items to that project, not elsewhere.
- A project with typed notes counts as "discussed" automatically — no separate flagging step.
- Un-discussing a project that has typed notes asks before discarding them, so a stray tap can't destroy notes.
- Open items and people moved into collapsible sections beneath the projects so the panel stays project-first without losing anything.

What you see today: Open any meeting → the desktop screen is split projects-left / notes-right (phone: a toggle at the top). Mark a project discussed and type into its card. When you End & Summarize, that project's tasks land on that project.

Why it matters: "Are the tasks actually going to tie to the correct project I was talking about?" was the biggest trust gap in the summarize flow. This removes the guesswork at the source — the notes arrive already sorted.

2026-06-08 — Pip Autonomous Operator: Pip works the book overnight (Phase 1)

What I built: Pip now works your accounts overnight on a schedule, instead of only thinking when you open the app. Each morning there's an "operator report" waiting on your Home screen — a prioritized plan for the day with follow-up emails already drafted, ready for you to review and send.

Problem it solves: Until now every smart thing Pip did happened the moment you opened a screen — Pip was reacting. The daily brief told you *what's happening*; you still had to go do all of it. This turns Pip from an advisor you consult into a chief of staff who's already started the work before you sit down.

What changed:

- A scheduled job runs each night and reviews your whole book. It only does the deep thinking on the accounts that actually moved since the last run — the quiet ones are skipped — so it stays cheap and fast.
- For each account that moved, Pip writes down where things stand, the risks, a drafted follow-up email where one's warranted, and suggested next moves. That work is saved, not thrown away.
- It all rolls up into one morning report on Home: the day's plan, with the drafted emails one tap from your clipboard or your mail app.
- **Nothing is sent and nothing is changed without you.** Everything Pip produces overnight is a draft you approve. Pip never reaches outside Folios — no email accounts connected, no customer data leaving the app.
- **It runs every morning, but only thinks hard about what changed.** The expensive per-account work only happens on accounts that actually moved since the last run, so the cost tracks activity, not the size of your book.

What you see today: On a morning after the loop ran, the top of your Home screen is the **Pip · Operator Report** instead of the usual daily brief — the prioritized plan plus a "Pip drafted N follow-ups" section with Copy / Open in Mail buttons. The same overnight work also shows up where you act on it: each **account screen** has a "Pip worked this overnight" panel (the situation, the drafted email, and proposed next steps you approve or dismiss one tap at a time), the **Cadence Hub** shows a pre-built agenda before a call, and the **Gauge** Pip card lists proposed moves across your whole book as

a quick approve/dismiss queue. On a quiet day (or before the first run), you get the normal live daily brief as before.

Why it matters: This is the leap from "an external brain you feed" to "a digital chief of staff that works the book." The relationship knowledge doesn't just sit in Folios — Pip acts on it for you, every night, and none of it walks out the door. The overnight work is computed once and read everywhere, so opening the account, the cadence, or Gauge shows pre-done work instead of spinning up a fresh think. (The only surface still on the list: projecting the plan onto the Calendar timeline, plus an optional phone notification when the report is ready.)

2026-06-06 — Teach Pip: build his knowledge on demand

What I built: A way to deliberately sit down and have Pip ask you a stream of questions to sharpen what he knows — instead of only getting a couple gentle ones a week.

Problem it solves: Pip learns your world from the questions you answer, but those came out slowly (a few a week) and capped out — so there was no way to say "I've got ten minutes, ask me everything." If you wanted to invest in making Pip smarter, you couldn't.

What changed:

- A **"Teach Pip about your world"** card on Home (and a **"Catch up with Pip"** button in Settings) opens a focused Q&A session any time — even when nothing's queued.
- Inside the session, a **"Pip, ask me more →"** button generates a fresh batch on the spot, lifting the normal once-every-6-hours / five-at-a-time limits because *you* asked. Answer, ask for more, repeat — for as long as you like.
- Every answer still flows into the same places (Pip's profile of you + your vocabulary glossary), so it compounds into better briefs and summaries.

What you see today: Click in, answer questions that are clearly drawn from *your* accounts (not a generic quiz), and tap "ask me more" whenever you want to keep going.

Why it matters: The more Pip knows, the more useful every brief is. This turns "teaching Pip" from a slow trickle into something you can actually invest in when you have the time.

2026-06-06 — Leadership tasks: to-dos from your 1:1s have a home

What I built: Action items from a 1:1 or internal/leadership meeting can now live as *your own* tasks, instead of being forced onto a customer account.

Problem it solves: Folios already let you run recurring 1:1s (with your manager, a teammate) that aren't tied to a customer. But when you summarized one, every action item wanted an account to file under — which is wrong for "send the forecast up to leadership" or "prep the partner-review deck." Those are your work, not an account's.

What changed:

- Items from a person/internal cadence that you *don't* route to an account now persist as **leadership tasks** — tagged to the 1:1 they came from, with no account.
- The summarize screen stops nagging you to route them: for a 1:1, an un-routed item reads "↳ My task · no account" instead of a yellow "route this somewhere" warning. (You can still route a specific item to an account if it belongs there.)
- Each 1:1's hub now lists its open leadership tasks with a one-tap done — so they don't disappear.

What you see today: Run your weekly 1:1, capture "follow up on headcount," leave it un-routed, and it's waiting for you in that 1:1's hub — not buried under some random account.

Why it matters: Your own commitments out of leadership meetings are some of the most important things you carry. Now they have a home instead of falling through the cracks.

2026-06-06 — Pip finishes the "chief of staff" build + you can edit your profile

What I built: The last pieces of Pip's portfolio intelligence, plus a place to edit what Pip knows about you directly.

Problem it solves: Pip could already brief you across the whole portfolio, but it couldn't notice when an account had quietly gone off your *own* usual rhythm, and it waited to be asked before drafting a follow-up. And the profile Pip builds from your answers was read-only — no way to correct it without answering more questions.

What changed:

- **Off-cadence radar.** The daily brief now flags accounts you've drifted away from *relative to how often you normally meet them* — "you usually meet every ~3 weeks; it's been 45 days" — not a generic "cold" threshold. It learns each account's rhythm from your own history.
- **Drafts waiting for you.** A "Pip drafted these follow-ups" card on Home surfaces meetings you wrapped a couple days ago that still have no follow-up logged — with the follow-up email Pip already wrote ready to copy or open in Mail. No extra cost; it reuses the draft from when you summarized.
- **Edit your profile.** Settings → "What Pip knows about you" now lets you edit the basics (role, company, industry, portfolio, goals, working style) inline, and re-run the intro interview anytime. Edits feed every brief, summary, and chat.
- **Sharper questions.** The weekly question generator now also reads your champions/blockers and recent meeting summaries, so the questions it asks connect threads across accounts instead of one-at-a-time clarifiers.

What you see today: Home tells you who's gone quiet by your own standard and hands you drafts to send; Settings lets you keep Pip's picture of you accurate.

Why it matters: This is the difference between an assistant that answers when asked and one that notices things and gets ahead of them.

2026-06-05 — Project status updates + multi-account project picker

What I built: Two upgrades to Gauge projects. First, a "status updates" pulse log on every project — a running, timestamped heartbeat separate from the durable notes field. Second, the account field on a project is now a searchable picker that lets you tie a project to several accounts.

Problem it solves: Project notes were one big blob, so there was no clean way to post "here's where this stands today" without burying the last status. And linking a project to its accounts meant scrolling a long checkbox list instead of just searching.

What changed:

- **Status updates.** The expanded project card shows the latest update with a relative timestamp ("Updated 2h ago"), plus a box to post a new one (Enter to post). Each post is stamped with the time and who wrote it, and the project edit screen shows the full history. Pip reads the latest update (and the prior two) so its briefs can say things like "All Star — latest: 'waiting on legal sign-off' (Jun 3)."
- **Searchable multi-account picker.** When building or editing a project, search for an account, click to add it as a chip, then search again to add more. Picked accounts drop out of the search so you can't add them twice, and chips remove with one tap.
- **Meeting agenda, surfaced.** When you open a meeting you scheduled with an agenda, that agenda now greets you at the top of the meeting sidebar instead of being hidden.
- **Pip knows what's coming.** Upcoming scheduled meetings now feed Pip's per-account context, so when you ask "what's on my plate for Acme?" it can mention the meeting you've booked.

What you see today: Projects read as a living status feed, link cleanly to every account they touch, and Pip's briefs reflect both the latest project pulse and your upcoming calendar.

Why it matters: A project's current state and the meetings ahead of you are exactly what a chief-of-staff needs at a glance — now Pip has both, and so do you.

2026-06-05 — Schedule future meetings on the calendar

What I built: A lightweight way to put a single upcoming meeting on a specific date without it being part of a recurring cadence. Click any empty day on the calendar, pick the account, set a date and time, add an optional agenda — and the meeting appears on the calendar, the list view, and the Home screen alongside your cadence events.

Problem it solves: The calendar only showed recurring cadence occurrences. If you knew you had a one-off call booked for next Thursday there was no way to put it on the calendar so Pip could surface it as "today's meetings" and fire reminders like it does for cadences.

What changed:

- A new Schedule Meeting modal (account picker, date/time, method, optional agenda) reachable from any empty calendar day click or the "+ Schedule Meeting" header button.
- Scheduled meetings appear in Calendar, Week, and List views with a `◆` chip distinct from cadence cards.
- Home screen surfaces a "Scheduled Today" section showing upcoming same-day scheduled meetings with a one-tap "Open →" to go straight into meeting mode.
- Reminders (30 min, 5 min, at start) fire exactly like cadence meeting reminders — including browser notifications if permission is granted.
- Opening a scheduled meeting on or after its day flips it to a live draft and opens the full-screen note-taking and summarize flow — no separate path needed.

What you see today: The calendar is now a complete picture of your planned meetings, not just the recurring cadence pattern.

Why it matters: When you leave a call knowing you need to follow up in two weeks, you can book the slot immediately instead of relying on a reminder app you may never check.

2026-06-05 — You can now see what you've told Pip

What I built: A place in Settings that shows everything you've told Pip and the profile it has built from your answers — plus a fix so your answers fold into that profile within minutes instead of the next day.

Problem it solves: Answering Pip's "Pip's Curious" questions felt like shouting into a void. The answers were saved and Pip was quietly using them in every brief, but there was no screen that showed them back to you — so a long session of answering looked like it did nothing.

What changed: Settings → Pip's Questions now shows "What Pip understands about you" (the narrative Pip builds from your answers) and "What you've told Pip" (your actual answered questions). And the background step that rebuilds the narrative from new answers now runs about five minutes after you answer, with a small confirmation, instead of once a day.

What you see today: Answer a few questions, and within minutes Settings reflects the updated understanding — and your raw answers are always listed there so nothing ever feels lost.

Why it matters: If the app asks you to invest a few minutes teaching it, it owes you visible proof that the investment landed.

2026-06-05 — Pip structured formatting + daily-brief fix

What I built: Pip now writes every brief and summary the way a person actually wants to read one — a headline, labeled sections, and bullets with the important names in bold — using its own small set of on-brand status icons instead of generic emoji. Also fixed the daily brief, which had started showing raw computer text.

Problem it solves: Right after the model upgrade, the daily brief rendered as a block of raw JSON (computer formatting) instead of readable text, and even when readable it was one long paragraph that was hard to scan. Pip's other write-ups were inconsistent — some structured, some a wall of prose.

What changed:

- **Daily brief fix.** The stronger model writes a fuller brief than the old one, which overflowed a size limit and caused the text to break into raw JSON on screen. Raised the limits on every relevant Pip endpoint so output completes cleanly, and added a safety net so the brief can never show raw code again even if something else goes wrong.
- **Structured formatting everywhere.** Chat, the daily brief, meeting summaries, Brief Me, the cadence pre-call brief, and the QBR all now render with headers, bullets, and bold — consistently.
- **Pip's own status glyphs.** A small custom icon set in Folios' visual language (needs-now, keep-an-eye, good-news, cross-account-pattern, done) marks priority sections, instead of clashy unicode emoji. Email drafts stay clean plain text.

What you see today: The Home daily brief reads as a tidy, skimmable rundown — a one-line headline, then short labeled sections with bullets and tappable account names. Meeting summaries and QBRs are slide-ready.

Why it matters: A brief you can read in five seconds gets read. This is the difference between Pip producing text and Pip producing something you actually act on.

2026-06-05 — Pip intelligence push (model tiering + structured suggestions)

What I built: A round of upgrades that make Pip noticeably sharper and let what you teach him actually change the data — safely.

Problem it solves: Pip had started to feel flat: nearly every Pip surface was running on the cheapest model, the "Pip's Curious" questions were generic or had quietly stopped appearing, and the things you told Pip (a contact's role, what a tool like "Fuse5" is) stayed as loose notes instead of updating the account.

What changed:

- **Model tiering.** The surfaces you actually judge Pip's intelligence by — the chat, meeting summarize, the daily brief, the questions Pip asks, the "who you are" profile, and the QBR — now run on the stronger Sonnet model. The mechanical, high-volume jobs (per-account Brief Me, drafting follow-up emails,

vocabulary extraction, memory compression, the boss readout) stay on the cheaper Haiku model. Each upgraded surface can be re-dialed from a setting without a redeploy.

- **Pip proposes, you approve.** When Pip asks about a contact's role, an account's objective, or a tool the account uses, your answer can be saved straight to that field with one tap — a pre-checked "Also save to…" toggle on the answer card. Untick it to keep the answer as a plain note. Nothing is ever changed silently, and it never touches account health or tier.
- **Systems an account uses.** Tools/systems (e.g. "Fuse5 is their inventory system") now show as chips on the account Overview and travel into every per-account Pip surface — so when "Fuse5" appears in raw meeting notes, Pip knows what it is instead of asking again.
- **Question pipeline fixed.** The generic filler questions are gone for good, and the portfolio question generator (which had silently jammed) is unblocked and tops the queue back up as you work through it.
- **Full knowledge coverage.** The glossary terms and your profile now reach the last two surfaces that were missing them — the daily brief and the leadership readout — so every Pip output speaks your vocabulary.

What you see today: Pip's chat and the questions he asks read sharper and more specific; the "Pip's Curious" card surfaces real, grounded questions and answering one can update the account in place; account screens show the tools each account runs on.

Why it matters: This is the difference between Pip *generating text* and Pip *knowing your world* — what you teach him now changes the data and shows up everywhere he helps.

> *Operator note: one schema migration backs this — `supabase/pip_structured_suggestions.sql` (adds `folio_accounts.systems` and `folio_pip_questions.suggestion`). Already applied to production and folded into `schema.sql`; additive and idempotent.*

2026-06-04 — Full audit-fix pass + schema reconciliation

What I built: A top-to-bottom audit of the whole app and fixes for everything it surfaced — correctness bugs, a couple of data-model gaps, security/RLS gaps, and theme/accessibility polish — with new automated tests so the same bugs can't silently come back.

Problem it solves: A pile of small, quiet failures: project labels rendering blank, status pills tinted black in light mode, "how many overdue items?" always answering zero, completed tasks not syncing between the queue and the project board, 1:1 meetings vanishing from their hub, the leadership view under-counting the team's work, and onboarding/aliases breaking on a fresh deploy because the canonical schema had drifted.

What changed:

- **Correctness:** Pip chat now actually counts items and tracks commitments; project chips show titles; completing a task syncs the project board; status pills tint correctly in both themes; person-1:1 meetings are kept with their account; monthly cadences on the 29th–31st stop drifting into the next month; contact engagement matches informal names ("Mike" → "Michael Smith").
- **Data model:** `schema.sql` is canonical again (folds in the profile, drip-question, contact-alias, theme, is_global, nickname, and relationship columns/tables); contact aliases now work for solo users; account "Updates" can be edited.
- **Security:** new additive RLS so org peers can see each other's tasks (powers the leadership + teammate views); team invites now require you to actually belong to the org.
- **Polish:** light-mode theme tokens, reduced-motion gating, date off-by-one fixes, and accessibility labels.

What you see today: The home screen, Pip chat, Gauge board, leadership view, and account screens all show accurate, in-sync data; light mode looks right; and a fresh deploy comes up clean.

Why it matters: This is the "never show stale or wrong data" foundation — the thing that makes the app trustworthy enough to run a book of business on.

> *Operator note: two SQL migrations must be run in Supabase for the new capabilities — `supabase/folio_tasks_org_read.sql` (leadership/teammate task visibility) and `supabase/folio_contact_aliases_user_scope.sql` (solo-user aliases). Both are additive and idempotent; also captured in `schema.sql`.*

2026-06-03 — PWA share sheet (Phase 1)

What I built: A Web Share Target so you can share text, links, or notes from any phone app directly into Folios.

Problem it solves: Every meeting note required sitting down and manually retyping content you'd already written elsewhere (email, Messages, notes app).

What changed: `share_target` added to `manifest.webmanifest` (via `vite.config.js`). New `ShareTargetView` route handles shared content, lets you pick an account, and opens it in the meeting flow with the text pre-filled.

What you see today: From your phone, tap Share on any note or email → Folios appears in the share sheet → pick the account → it opens in a meeting note with the text ready.

Why it matters: Field AMs write notes everywhere in the moment. This closes the retyping loop.

2026-06-03 — Voice dictation in meetings

What I built: A "Dictate" mic button in the meeting notepad that transcribes speech directly into your notes.

Problem it solves: Typing notes while driving between calls is unsafe and slow. Voice capture lets you dictate a quick recap hands-free.

What changed: Dictate button added to CadenceMeetingMode (the full-screen meeting overlay). Uses the browser's built-in Web Speech API — no backend, no cost. Appends transcribed text as bullet points to the existing notes.

What you see today: In any meeting, tap the 🎙 Dictate button → speak → text appears in your notes as bullets. Tap ■ Stop when done.

Why it matters: Removes the biggest in-car friction point. You can debrief immediately after leaving a meeting.

2026-06-03 — Ask Pip recall polish

What I built: Quick-answer chips on the Ask Pip screen and a sharper "external brain" framing.

Problem it solves: The Ask Pip screen looked like a generic chat widget. Users didn't immediately know what to ask or how to use it as an external brain.

What changed: Greeting updated to "What do you need to remember?". Four quick-tap chips appear before you type (Recap last meeting, What did I promise?, What's at risk?, Who haven't I contacted?). Facts Pip knows about you are now prominently labeled in Settings as "What Pip knows about you".

What you see today: Open Ask Pip → see the chips → tap one → instant answer. No need to figure out what to type.

Why it matters: Positions Pip correctly as a memory layer, not just a chat window.

2026-06-03 — Stakeholder / relationship layer

What I built: Champion/blocker/neutral tags on each contact with a one-line "why" note.

Problem it solves: Pip had no structured sense of who controls decisions at each account — it had contact names and roles but not relationship dynamics. Briefs and

QBRs didn't lead with the power map.

What changed: Two new columns on `folio_contacts` (`relationship_role`, `relationship_note`). Contact cards show colored CHAMPION/BLOCKER pills. Pip context and summarize both emit a RELATIONSHIPS block so every brief and QBR leads with champion + blocker dynamics. SQL: `supabase/stakeholder_layer.sql` — run in production.

What you see today: Open any contact, tap "☆ Role", tag them as Champion/Blocker/Neutral with a one-liner. Pip's next brief will cite the power map directly.

Why it matters: "Sarah is the champion but Mike is the blocker" is the most important context in any sales situation. Now Pip knows it and leads with it.

2026-06-03 — Commitment enforcement

What I built: A daily commitment nudge on the home screen that monitors tasks you've marked as commitments (✦) and warns you 2 days before they're due — or flags them as overdue.

Problem it solves: Promises to clients would slip because there was no proactive reminder before the due date passed.

What changed: New `useCommitmentNudges` hook (client-side, zero LLM cost). HomeView shows an amber nudge card for the most urgent upcoming commitment with Mark Done and Snooze actions.

What you see today: If you have a ✦ commitment task due in ≤2 days, an amber card appears on the home screen above Pip's drip questions.

Why it matters: AM credibility lives or dies on follow-through. This closes the loop without you having to remember to check.

2026-06-03 — Meeting discussed signal — Pip now knows what you talked about

What I built: Two interlocked features that fix Pip's #1 failure mode: creating duplicate tasks instead of updating the ones you already discussed.

Problem it solves: During a meeting, you might talk through three open Gauge projects and two open action items. Pip gets your notes as free text and has to guess which of your active projects and items you actually touched. It defaults to creating new rows when it's unsure — which is frustrating when you clearly discussed "the LKQ integration project" and Pip creates a new task instead of updating the existing one.

What changed:

1. **Tap to mark discussed (Part A).** During a meeting in the full-screen meeting mode, you can tap any project card or open-item row in the sidebar to flag it as "discussed this meeting." Flagged cards show a teal ✦ Discussed chip and a teal left border. When you hit End & Summarize, those flagged IDs are sent to Pip as a high-confidence signal — Pip strongly prefers updating or closing flagged items rather than creating new ones.
2. **Live note highlight (Part B).** As you type your meeting notes, the sidebar automatically detects when you've written a project or item title in the notes and highlights the matching card with a subtle teal glow. This gives you instant visual confirmation that Pip will connect the dots — before you even hit summarize. No tapping required for the auto-detection.
3. **Plan modal badge.** In the summarize-preview modal, any row that Pip derived from a flagged project or item shows a ✦ Discussed badge so you can see Pip acted on your signal.

What you see today: In meeting mode, sidebar project cards now have a small "◇ Mark discussed" button in the top-right corner. Tap it and it turns to "✦ Discussed" in teal. Open-item rows are tappable directly to toggle the same flag. As you write notes, any card whose name appears in your text gets a soft teal glow. When you summarize, Pip's plan modal shows ✦ Discussed badges on the rows that connect to things you flagged.

Why it matters: This directly fixes the user's #1 complaint. When Pip knows what you discussed, it stops creating duplicates and starts doing what you actually want —

updating the right project, closing the resolved item, and routing the work correctly. The signal is explicit (you tapped it) so Pip gets the highest possible confidence, not a guess.

2026-06-03 — Pip drip questions (Phase 2)

What I built: After the Phase 1 onboarding interview, Pip now keeps learning through a gentle weekly drip. A "Pip's Curious" card appears on your Home screen with one question at a time — you type your answer inline, hit Answer, and Pip files it away. No modal, no interruption.

The problem it solves: The 5-question interview gave Pip a solid foundation, but a real relationship develops over time. Contacts gain roles, accounts get objectives, and your company builds up its own vocabulary of brands, programs, and codenames that no onboarding script could anticipate. Phase 2 is how Pip catches up to your real world without asking you to fill out a form.

What changed:

- **Daily gap detection (`detectKnowledgeGaps`)** — runs once per calendar day, zero LLM cost. Scans your data for three structural holes: contacts who appear in ≥3 meetings with no role on file, active accounts older than 30 days with no objective, and empty profile slots (if onboarding is marked done). Inserts `folio_pip_questions` rows (`source='gap_observed'`) for each gap found, templated from real names and account data.
- **Evergreen question bank** — 15 get-to-know-you questions (what a great week looks like, what metric you're judged on, which account keeps you up at night, etc.). Seeds only when the gap detector comes up empty, so the drip never stops.
- **Throttle (DB-driven, cross-device)** — max 1 per day, max 3 per rolling 7 days, 48h cooldown after any skip or dismiss. All persisted in `folio_pip_questions.answered_at` so it follows you across devices.
- **HomeView "Pip's Curious" card** — appears between the Daily Brief and the four-panel grid. Inline textarea (16px, no zoom), Answer / Skip / Not now buttons. Disappears when answered.

- **Terminology scan (`api/detect-terminology.js`)** — one Haiku call per week. Scans the last 30 meeting notes for proper nouns appearing ≥3 times that aren't a known account, contact, or glossary entry. Creates `category='terminology'` drip questions. When you answer, the answer writes to `folio_pip_facts` automatically.
- **Re-synthesis trigger** — when you've answered ≥3 drip questions since the last synthesis, a background call to `/api/profile-synthesis` updates your profile prose.
- **Settings → "Pip's Questions"** — global pause toggle and a 0–100% completeness progress bar.

What you see today: On home load, if Pip has a gap question queued and the throttle allows it, the "Pip's Curious" card appears. Answer it, skip it, or dismiss it. Over the first few weeks, Pip builds up your company's vocabulary and fills in missing data — silently, from data it already sees.

Why it matters: A good analyst doesn't interview you once and coast. Phase 2 is what turns Pip from a static profile into a system that gets smarter the longer you use it.

2026-06-01 — Pip onboarding interview + profile prose injection (Phase 1)

What I built: When you first sign in to Folios with no accounts yet, Pip now walks you through a short 5-question interview to learn who you are and how your business works. Your answers get synthesized into a short profile narrative that Pip reads before every response — briefings, meeting summaries, chat answers — so everything Pip says is grounded in your actual world, not generic advice.

The problem it solves: Pip is only as useful as the context it has. Without knowing your role, your company, how many accounts you carry, or what a good quarter looks like for you, Pip's outputs were technically correct but sometimes oddly generic. Now Pip knows the basics from the start.

What changed:

- Two new tables: `folio_user_profile` (one row per user, stores structured profile fields and a 4–8 sentence `profile_prose` narrative) and `folio_pip_questions` (the question queue and answer log). Both have RLS scoped to your user ID. SQL: `supabase/folio_user_profile.sql`.
- New full-page `PipOnboardingView`: 5 questions, one at a time, in Pip's voice. Resumable — if you close and come back, Pip picks up where you left off. Skippable at any point via "Finish later".
- New `/api/profile-synthesis` endpoint: takes your Q&A pairs, runs a single cheap Haiku call (~$0.002 once), and compresses them into a structured profile
  - a narrative paragraph that Pip reads going forward.

- New `useUserProfile` hook — reads and writes your profile row.
- `profile_prose` injected into both Pip entrypoints: meeting summarize (`pip.js`) and Ask Pip chat (`api/pip.js`). Both paths now prepend a "WHO YOU ARE" block to Pip's context so the same grounding applies everywhere.

What you see today: New users (no accounts) are routed directly to the interview screen on first login. Existing users (who already have accounts when this shipped) see a dismissible "Pip · Just for you" card on the Home screen with "Let's go →" and "Maybe later" options. The interview is soft-gated — you can always skip or finish later, and Pip still works fine without it.

Why it matters: This is the foundation for everything else in the "Pip knows my world" roadmap. Once Pip knows who you are, every brief, summary, and suggestion can be grounded in your specific role, industry, portfolio shape, and goals rather than generic account management advice. Phase 2 (the gentle weekly drip of gap-filling questions) builds on this base.

2026-05-31 — Gauge template total turnaround time

What I built: Templates in Gauge now carry an "Estimated Duration" field that tells you upfront how long a project type typically takes. When you create a project from a template, Gauge automatically sets an expected completion date so you always know roughly when the work should wrap.

The problem it solves: Until now, templates were a list of tasks with no sense of time. You'd pick a template and have no idea if it represented a 2-day task or a 6-week project. The expected completion date also gives Pip something concrete to flag — it can surface projects approaching or past their estimated finish.

What changed:

- Templates have a `total_duration_days` field. The TemplatePickerModal shows "Est. Xd" next to each template name when it's set.
- Templates with stage `due_offset_days` values auto-derive the duration as a placeholder (max offset across all stages). Manual override wins.
- When you create a project from a template with a known duration, `expected_complete_date` = today + duration days is set automatically.
- Project cards show the expected complete date in the meta row. Past-due expected dates show in amber so they're easy to spot.

What you see today: The "Est. Xd" chip on templates in the picker, and an "Est. complete · [date]" line on project cards built from timed templates.

Why it matters: Gives Chris an instant sanity check when picking a template ("this is normally a 14-day project") and surfaces an early warning when work is running long.

2026-05-30 — Gauge V3 Phase 6: polish + V2 brain wiring

What I built: Four cleanup items that tie the whole Gauge V3 build together and finish wiring Pip's learning loop into every Gauge surface that touches a task.

The problem it solves: Phases 1–5 left a few corners loose: edits made directly in the project stage editor or the kanban board weren't feeding Pip's correction log, the wrong-account-from-Pip case had no fix path after the plan had already been applied, AMs had no place on the home page to see "my projects" at a glance, and assignment hints only applied per-account so Pip couldn't generalize "Sara owns invoice work" across the org.

What changed:

- Corrections wiring — task-text edits made through the project stage editor, the standing-projects kanban, or the My Queue panel now all log to Pip's correction log the same way edits through the task detail panel always did. The V2 brain learning loop is now closed on every Gauge entry point.
- Post-apply account override — if Pip routes a task to the wrong account and you only notice after the fact, you can change the Linked Account field on the task and Pip logs that as a `routed_account_changed` correction so it learns the right home for next time.
- "Projects I own" rollup on Gauge home — for AM-lens users, a compact section appears above the stats row showing every active project on accounts they own (progress bars, status pills, account chips). Click any row to scroll to it in the list below.
- Org-wide assignment hints auto-promotion — once 3 different accounts have logged the same kind of work going to the same person (e.g. invoice tasks → Sara), the system promotes that into a cross-account hint so Pip routes future invoice work to Sara on any account, not just the three she's been doing it on.

What you see today: Same Gauge layout, but Pip is now learning from every edit and the AM view has a cleaner home rollup. The assignment-hint promotion is silent — you just notice over time that Pip starts routing the right work to the right person without you having to correct him every time.

Why it matters: This closes out Gauge V3. The V2 brain is now fully wired through Gauge (Phases 5 + 6 together), and the system is set up to actually get smarter the longer you use it.

2026-05-30 — Gauge V3 Phase 5: Leader view

What I built: The org-wide project rollup that the Leader lens has been waiting for, plus a read-only drill-in to see what any teammate has on their plate.

The problem it solves: Leaders need a view that's about *the team*, not their personal queue. Until today, leaders were looking at the same AM-focused project list as everyone else — useful for their own work, useless for spotting that Tony's been stuck on the KSI Salvage audit for 12 days.

What changed:

- New "Leader" tab in Gauge (only visible to users whose default view is Leader). Lists every project across every AM and account, hiding drafts and completed work.
- Each row shows: title, status, account chip, AM chip, progress (X/Y stages, percent), due date, and a "STUCK · Nd" pill that lights up when no stage has been completed in 7+ days. Expanding a row shows every stage with its assignee and due date inline.
- Filter bar: by AM, by account, by status, by stuck-only. Sort by due date, progress, or stuck-time. "Clear" resets all.
- Clicking an AM chip drills into a read-only view of that teammate — their open tasks, project stages assigned to them, projects they're on, and the accounts they touch. No edit buttons; the point is visibility, not mutation.
- Open-project button on each expanded row jumps over to the standard Projects view with that project's row pre-expanded.

What you see today: If your default view is Leader (set on the team-member record), Gauge opens straight to the Leader rollup. Everyone else still lands on Projects or Tasks. Toggle in the top-left switches between Leader / Projects / Tasks.

Why it matters: This is the team-rollup story the Leader lens from Phase 2 was setting up. Pip's tone already adapts per lens; now there's a real UI to match. The drill-in pattern also lays groundwork for org-wide hints in Phase 6 — once you can see what Sara owns, the system can start routing similar work to her.

2026-05-30 — Gauge V3 Phase 4: discrete project templates

What I built: Save any project as a template, then spin up new projects from it in one click — with assignees pre-filled and due dates auto-scheduled relative to the day you create the new project.

The problem it solves: Discrete projects like audits or onboardings have the same stages every time — same people, same approximate durations. Today you'd re-type all 7 stages every time. Templates remove that drudgery.

What changed:

- "Save as template" button at the bottom of the project modal now preserves each stage's assignee email and a "due offset days" value (how many days after kickoff each stage is due).
- "+ From Template" picker (already in Gauge) now hydrates due dates when you use a template — every stage gets a due_date computed from today + the saved offset. Assignees pre-fill from the saved emails.
- Sub-stages get the same treatment.

What you see today: Saving a project as a template now actually captures the structure people use — who does what, when. Using a template puts a fully populated project on the table ready to tweak, not a blank skeleton.

Why it matters: Templates were partially built earlier (the table

- basic picker existed) but never carried the data that made them worth using. Now they do. AMs running repeat workflows save real time.

2026-05-30 — Gauge V3 Phase 3: flat task queue

What I built: The new flat task queue — every task and action item across every account and project in one scannable list. Toggle between Projects and Tasks at the top of Gauge.

The problem it solves: Before today you could see projects, but you couldn't see "everything on my plate right now" without bouncing between accounts. Tasks lived inside projects, which were inside accounts, which were inside workspaces. Too many layers of indirection for the simple question "what am I supposed to do?"

What changed:

- New "Tasks" tab at the top of Gauge, alongside "Projects." Renders a flat queue of every task and item, sorted by due date.
- Cards show the task title big, with account chip + project chip + due date underneath. Discrete projects get a "Step 3 of 7" badge so admins know where they are in the larger thing.
- Three sub-filters: Open (default), Mine (filters to tasks assigned to you), All (everything).
- "Group by project" toggle clusters tasks by their parent project.
- Admin lens lands directly on Tasks tab by default. AM and Leader land on Projects (current behavior preserved).
- A one-time SQL backfill brings existing items and project stages into the new unified table so the queue isn't empty on first load.

What you see today: A Projects | Tasks toggle at the top of Gauge. Click Tasks to see the new flat queue. Pip's plan-apply path has been dual-writing since this morning, so newly-created tasks land immediately; existing tasks land after running the backfill SQL.

Why it matters: This is the first user-facing surface of Gauge V3. The lens system from Phase 2 quietly defaults Admin users to the queue. Phases 4-6 add project templates, the Leader rollup, and the polish layer that ties the system together.

2026-05-30 — Gauge desktop polish: capped list + insights sidebar

What I built: Capped the desktop project list width so cards stay scannable, and put a "Pip + insights" panel in the right margin so the empty space turns into a steering wheel.

The problem it solves: On a wide monitor, project cards stretched edge-to-edge. Your eye had to scan from left to right to read one row — the same content felt harder to read than on mobile, where the narrow cards stacked cleanly.

What changed:

- Project list is now capped at 720px wide on desktop, single column, left-aligned. Matches the cohesive feel of the mobile view.
- Right sidebar (desktop only) shows three blocks: Pip's notice (moved from above the list), a "Stuck" list of in-progress projects whose updated_at is more than 7 days old, and a "Team load" tally of who has the most open task items across all live projects.
- Mobile view is unchanged.

What you see today: A cleaner, narrower project list on desktop with a steady sidebar to the right showing what's stuck and who's loaded up. Numbers and names are derived live — no new data to maintain.

Why it matters: Same data, way easier to scan. The same pattern (capped content column + sidebar of derived insights) will land on Accounts and other list views over time. Phase 3 of Gauge V3 (the real flat task queue) will inherit this layout instead of relearning it later.

2026-05-30 — Gauge V3 Phase 2: lens system

What I built: The plumbing for the three role-based views Folios will use going forward — **AM**, **Leader**, and **Admin**. Each member of a team now has a default view assigned at invite time.

The problem it solves: Different people in the org care about different things. An account manager wants to see *their accounts*. A leader wants to see *what the team is up to*. An admin wants to see *what they need to finish*. One UI showing all three at once would be cluttered for everyone.

What changed:

- New `default_lens` column on team members in the database (AM / leader / admin). Existing members were backfilled — owners and admins got Leader, everyone else got AM.
- The invite modal now has a "Default view" dropdown next to role. It smart-prefills based on the role you pick (Leadership → Leader, Member → AM) but you can override.
- Pip now knows what view each user lives in and frames his answers accordingly — strategic team rollup for Leaders, account-focused for AMs, execution-mode for Admins. Same Pip personality, different altitude.

What you see today: A "Default view" dropdown when you invite a new team member. Pip's answers in chat already feel slightly different per lens. No new UI surfaces yet for Leader or Admin views — those land in Phases 3-5.

Why it matters: Pip's tone now adapts to who's asking, which is the first real lens-aware behavior in the product. Phase 3 builds the actual queue UI; Phase 5 builds the Leader rollup. Phase 2 is the data foundation that makes both possible.

2026-05-30 — Gauge V3 Phase 1: unified task home

What I built: A single new database table called `folio_tasks` that will eventually hold every task and action item in Folios.

The problem it solves: Before today, your tasks lived in two different places. Loose action items went into one table; tasks tied to projects went into a different one (nested inside the project itself). Pip had to guess which bucket to use, and you couldn't see all your work in one queue. The plumbing was fundamentally split.

What changed: New unified `folio_tasks` table exists in production. Pip now writes to both the old places (so nothing breaks for you today) AND the new unified place (so the new queue UI has real data to read from when it lands).

What you see today: Nothing. This was pure plumbing — no UI changes. But every Pip plan you Apply going forward is silently populating the new table.

Why it matters: This is the foundation for the three-lens views (AM / Leader / Admin) and the flat task queue coming in Phases 2-6. Without one home for tasks, none of that could exist.

2026-05-31 — Pip Tier A: portfolio intelligence + daily brief

What I built: Pip can now see your whole portfolio at once. Every morning it takes a snapshot of every account's health — how cold each one is, how many items are overdue, which projects are moving and which are stuck — and then writes you a plain-English brief on the home screen.

The problem it solves: Before today Pip only knew about one account at a time. If you opened ACME's cadence hub, Pip knew everything about ACME. But it had no idea that Parts Authority was 45 days cold, or that three accounts had overdue items piling up. You had to hold the full picture in your head. Pip now holds it for you.

What changed:

- New `folio_account_snapshots` database table. One row per account per day, computed from real signals: health status, days since last contact, open/overdue item counts, active/stuck Gauge project counts. Zero AI cost — pure arithmetic.
- Snapshots are computed once per calendar day on app load, silently, in the background. They never block the UI.
- Daily brief card on the home screen. One Haiku call, cached for the whole day (not re-run until tomorrow). Pip reads the snapshot table and writes a 3-5 sentence morning read: what needs attention, biggest risk, any wins worth noting.
- `buildPortfolioState()` utility that compresses the portfolio into a short text block — the building block for the 1:1 mode and boss-ready rollup coming in Tiers B-D.

What you see today: A "Pip · Daily Brief" card at the top of the home screen, just above the four panels. It loads a few seconds after the home screen appears (while Pip

is thinking). The next day it shows instantly from cache. If there are no snapshots yet (first app load after this upgrade), the card stays hidden.

Why it matters: This is the Tier A foundation for Pip's "chief of staff" mode. Every more sophisticated portfolio feature — people modeling, pattern detection, proactive briefings for your boss — runs on top of these daily snapshots. The snapshots are free to compute and cheap to query. The brief costs roughly $0.07/month at Haiku pricing.

Earlier upgrades

Pre-2026-05-30 upgrades happened before this log existed. They're captured in changelog.md and the "Already shipped" section in CLAUDE.md (internal working doc). Going forward, every major upgrade lands here in plain English.